

第六章 数据的机器级表示

1. **比特(bit)**: 信息最小单位 (每一个 0 或 1), 也叫**位**或**二进制位**。接近 0 的电压视为 0, 远离 0 的电压视为 1。

2. 位组合: 有 k 位, 则区分 2^k 个值。

3. 按位运算

① **按位与运算 &**

位屏蔽 (掩码): 屏蔽位 0, 凸显位 1。

② **按位或运算 |** (同或运算, 包含或运算)

③ **按位非运算 ~** (补运算)

按位取反

④ **按位异或运算 ^**

相异为 1, 相同为 0

按位取反: 取反位 1, 保持位 0

判断相同: 两个数运算结果每一位都等于 0 则相同。

⑤ **左移 <<**

右侧空位用 0 补充

⑥ **算术右移 >>**

左侧空位符号扩展

逻辑右移: 左侧空位用 0 补充

4. **布尔代数**: 二进制逻辑运算。

与 $A \cdot B$, 或 $A+B$, 非 $\bar{A}$, 异或 $(\bar{A} \cdot B) + (A \cdot \bar{B})$

① 恒等律: $A+0=A$, $A \cdot 1=A$

② 0/1 律: $A+1=1$, $A \cdot 0=0$

③ 互补律: $A+\bar{A}=1$, $A \cdot \bar{A}=0$

④ 交换律: $A+B=B+A$, $A \cdot B=B \cdot A$

⑤ 结合律: $A+(B+C)=(A+B)+C$, $A \cdot (B \cdot C)=(A \cdot B) \cdot C$

⑥ 分配律: $A \cdot (B+C)=(A \cdot B)+(A \cdot C)$, $A+(B \cdot C)=(A+B) \cdot (A+C)$

⑦ 德摩根律: $\overline{A \cdot B} = \bar{A} + \bar{B}$

5. **数据类型**: 计算机对信息的编码表示法。

6. **无符号整数**: 有 k 位, 表示 $0 \sim 2^k-1$

7. **有符号整数**

	原码	反码	补码
正数 (首位 0)	不变	不变	不变
负数 (首位 1)	首位变 1	按位取反	按位取反后加 1 / 右侧若干 0 和第一个 1 不变, 其他位取反

8. **计算机采用二进制补码表示有符号整数**。有 k 位, 能表示 $-2^{k-1} \sim 2^{k-1}-1$

1000...: 绝对值最大的负数 (最大正整数的相反数-1)

1111...: -1

0000...: 0

$2^0=1$ $2^4=16$ $2^8=256$ $2^{-1}=0.5$

$2^1=2$ $2^5=32$ $2^9=512$ $2^{-2}=0.25$

$2^2=4$ $2^6=64$ $2^{10}=1024$ $2^{-3}=0.125$

$2^3=8$ $2^7=128$ $2^{11}=2048$ $2^{-4}=0.0625$

9. **ALU: 算术逻辑单元**

10. 保证 ALU 正确做加法的运算原则

① 绝对值相同符号相反的两个整数相加得 0。

② 每个数加 1 得到正确结果。(首位的进位总是被忽略)

11. 二进制转化为十进制: 按各位权重相加。

12. 十进制转化为二进制

整数部分：除 2 取余，由下到上。

小数部分：乘 2 取整，依次排列。

例：45.8125₁₀=0101101.1101₂=1.011011101X2⁵₂

13.算数进位

源 A+源 B+ carry in	0	1	2	3
结果位	0	1	0	1
carry out	0	0	1	1

14.符号扩展 (SEXT)

正数以 0 向左扩展

负数以 1 向左扩展。

不同长度补码整数作加法运算需先符号扩展。

15.溢出：加法结果超出表示范围。

表现形式：两个正数相加得负数，两个负数相加得正数。

一个正数与一个负数相加永不溢出。

16.ASCII 码：美国标准信息交换码，所有计算机处理单元和输入输出设备转换字符的标准码。

键盘上每个键被唯一 ASCII 码识别，8 个二进制位表示，首位为 0。

0-32 号、127 号：通讯专用字符或控制字符。

33-126 号：可见字符。

17.定点小数小数点对齐作加法。

18.浮点数用来表示精度少、位数多的小数。

19.IEEE：国际电气和电子工程师协会

20.浮点数算数运算标准：IEEE-754 浮点数。

21.单精度浮点数

范围：10^{±38}

精度：7 位

符号 s (1 位，0 正 1 负)+指数域 exponent (8 位无符号整数)+分数域 fraction (23 位)

$$\begin{cases} (-1)^s \times 1.fraction \times 2^{exponent-127}, & 1(00000001) \leq exponent \leq 254(11111110) \\ (-1)^s \times 0.fraction \times 2^{-126}, & exponent = 0(00000000) \\ (-1)^s \infty, & exponent = 255(11111111), fraction = 0 \\ NaN(not a number), & exponent = 255(11111111), fraction \neq 0 \end{cases}$$

22.双精度浮点数

范围：10^{±306}

精度：15 位

符号 s (1 位，0 正 1 负)+指数域 exponent (11 位无符号整数)+分数域 fraction (52 位)

23.浮点数运算步骤

- ①使指数相等
- ②分数运算
- ③结果规格化
- ④丢失位舍入
- ⑤判断指数溢出

24.十六进制和八进制方便人们手工处理二进制位。

25.一个十六进制位对应四个二进制位

0000=0	0100=4	1000=8	1100=C
0001=1	0101=5	1001=9	1101=D
0010=2	0110=6	1010=A	1110=E
0011=3	0111=7	1011=B	1111=F

26.十六进制字面常量书写时使用前缀 0x 或 x，在 C 语言中十六进制字面常量使用前缀 0X 或 0x。

27.一个八进制位对应三个二进制位

000=0	010=2	100=4	110=6
-------	-------	-------	-------

001=1 011=3 101=5 111=7

28.字符串：字符序列，以 NUL（0）结束

29.图像：像素矩阵

{	黑白：1 位，0 白 1 黑
	彩色：RGB(red,green,blue)各 8 位

30.声音：定点数序列

31.C 语言中的数据类型采用位数与计算机指令集结构和编译器有关。

int：二进制补码整型

char：ASCII 码（进行运算时与对应 ASCII 码等值）

double：双精度浮点数

32.格式说明

%d 十进制补码整数

%x 十六进制

%o 八进制

%c 字符

%f 单精度浮点数

%lf 双精度浮点数

第七章 数字逻辑电路

1. **集成电路**：用工艺将电路中的元件及布线互连，固定在半导体晶片或介质基片上，封装在管壳内，称为具有所需电路功能的微型结构。

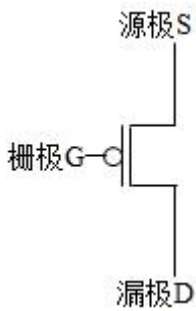
2. **摩尔定律**：集成电路上可容纳的晶体管数目约每隔 18 个月增加一倍。即，价格不变时，每一美元能买到的电脑性能每隔 18 个月翻两倍以上。

3. 微处理器由 **MOS（金属氧化物半导体）晶体管** 组成。

4. **N 型 MOS 晶体管源极不能接电源正极**。栅极加 2.9V 电压，漏极和源极导通；栅极加 0V 电压，漏极和源极断开。

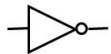


5. **P 型 MOS 晶体管源极不能接地**。栅极加 2.9V 电压，漏极和源极断开；栅极加 0V 电压，漏极和源极导通。

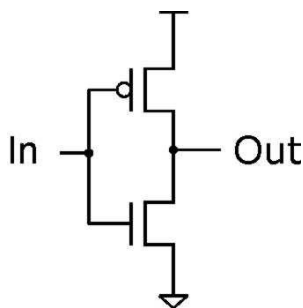


6. **CMOS（互补金属氧化物半导体）电路**：既包含 P 型晶体管，又包含 N 型晶体管。

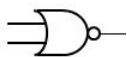
7. **反相器（非门）**



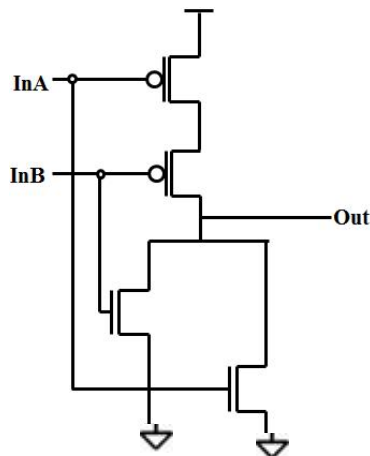
In	Out
1	0
0	1



8. **非或门**



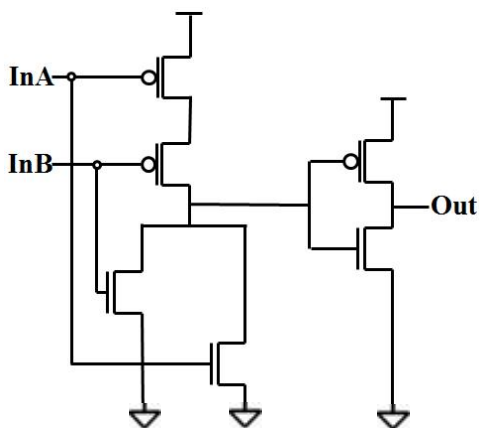
InA	InB	Out
1	1	0
1	0	0
0	1	0
0	0	1



9.或门（非或门+反相器）



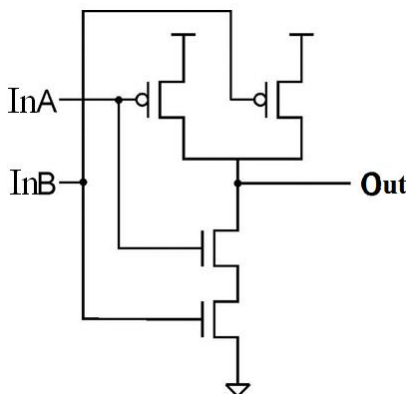
InA	InB	Out
1	1	1
1	0	1
0	1	1
0	0	0



10.非与门



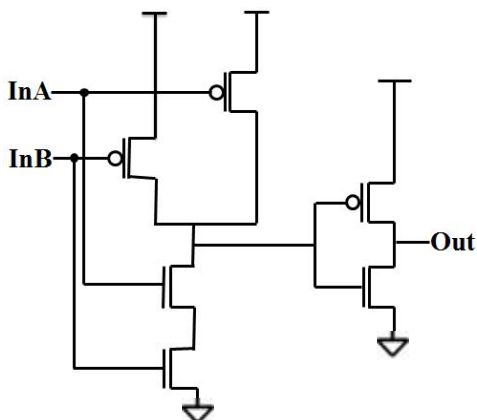
InA	InB	Out
1	1	0
1	0	1
0	1	1
0	0	1



11.与门（非与门+反相器）



InA	InB	Out
1	1	1
1	0	0
0	1	0
0	0	0



12.组合逻辑电路（判定元件）不存储信息，输出值只由当前输入值决定。包括译码器、多路选择器、全加法器。

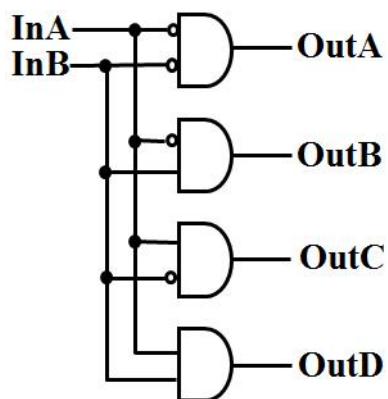
13.译码器

只有一个输出为 1，其他全为 0。若有 n 个输入，则有 2^n 个输出。

作用：判断某个位组合。

$n=2$ ，2-4 译码器：

InA	InB	OutA	OutB	OutC	OutD
1	1	0	0	0	1
1	0	0	0	1	0
0	1	0	1	0	0
0	0	1	0	0	0



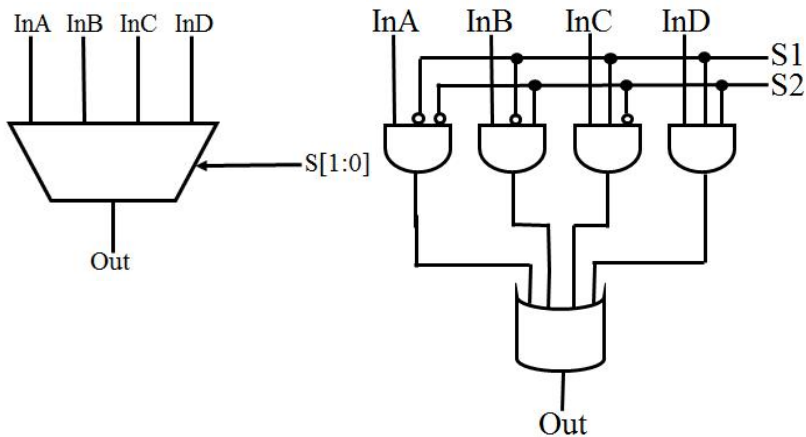
14.多路选择器

由选择信号决定哪个输入连接到输出。

n 条选择线, 2^n 个输入, 1 个输出。

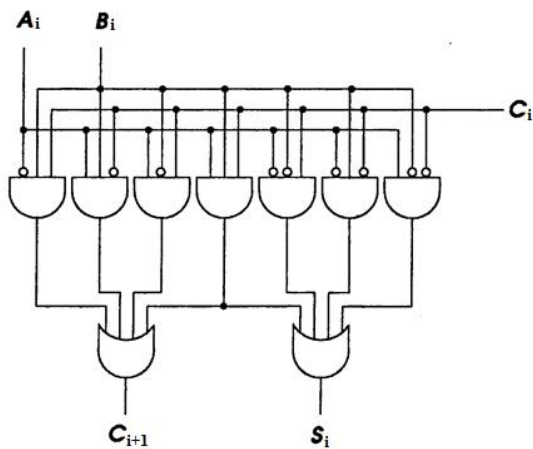
$n=2$, 4-1 多路选择器:

S1	S2	Out
1	1	InD
1	0	InC
0	1	InB
0	0	InA

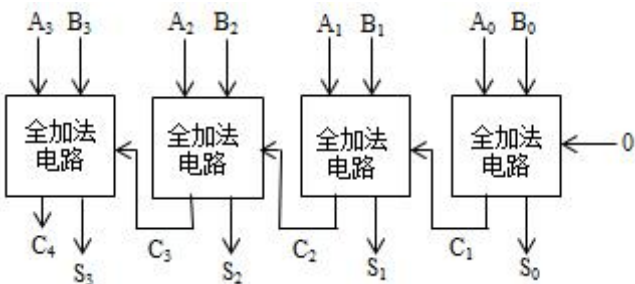


15.全加法电路

In			Out	
a_i	b_i	c_i	c_{i+1}	s_i
0	0	0	0	0
0	1	0	0	1
1	0	0	0	1
1	1	0	1	0
0	0	1	0	1
0	1	1	1	0
1	0	1	1	0
1	1	1	1	1



4 位加法电路:

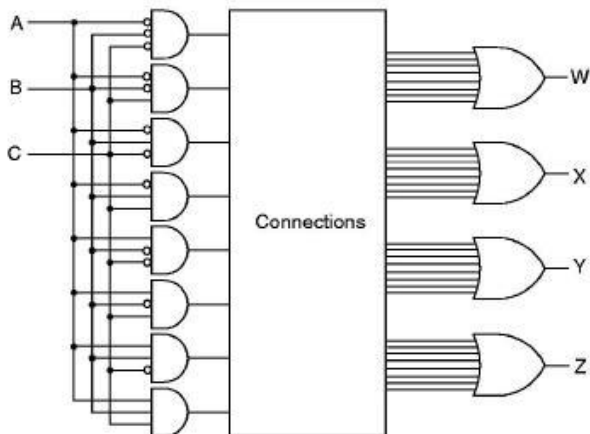


16.PLA (可编程逻辑阵列): 可以实现任意逻辑运算的通用组件。

与阵列: 每个与门有 n 个输入, 一共 2^n 个与门。

或阵列: 输出。

$n=3$:



17. **逻辑完备性**：提供足够多的与或非门，可以实现任意逻辑运算。即，门集合{与、或、非}在逻辑上完备。

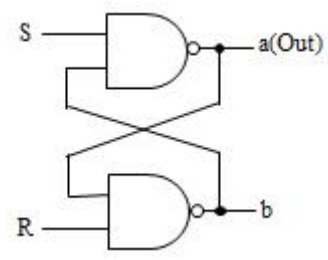
18.任意逻辑运算可以通过 PLA 实现。

19.**基本存储元件**：能够存储信息的元件。包括 **R-S 锁存器**、**门控 D 锁存器**、**寄存器**。

20.**R-S 锁存器**：能存储一位的信息。

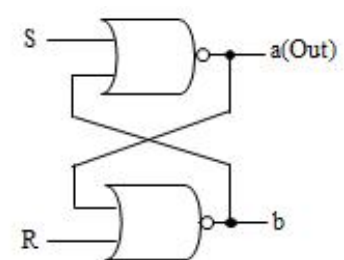
①非与门：

S	R	状态
1	1	静止状态， a 与 b 相异，锁存 a 值
1	0	存储 0（清空）， a=0, b=1
0	1	存储 1， a=1, b=0
0	0	取决于晶体管电子特性（禁止此操作）



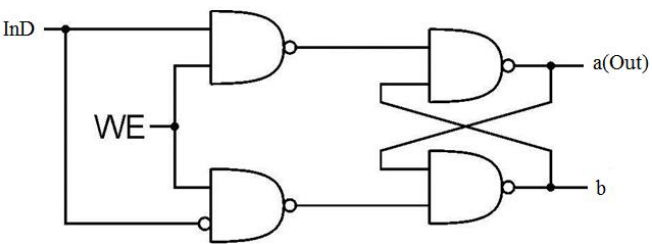
②非或门：

S	R	状态
1	1	静止状态， a 与 b 相异，锁存 a 值
1	0	存储 1， a=1, b=0
0	1	存储 0（清空）， a=0, b=1
0	0	取决于晶体管电子特性（禁止此操作）



21.门控 D 锁存器

InD	WE	状态
1/0	0	静止状态， a 与 b 相异，锁存 a 值
1	1	存储 1， a=1, b=0
0	1	存储 0（清空）， a=0, b=1

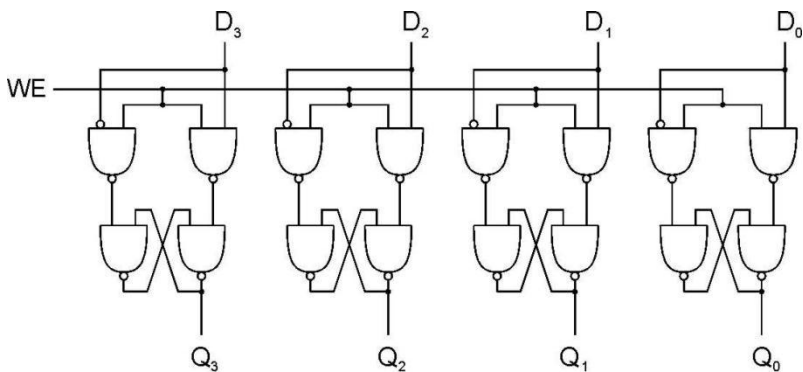


优点：对 R-S 锁存器何时设为 1 和 0 进行控制，避免 R=S=0 的情况。

22.寄存器

多个门控 D 锁存器存储多位，共享 WE。

4 位寄存器 Q[3:0]：



23.存储器由一定数量的单元组成，每个单元可被唯一识别，都有存储一个数值的能力。

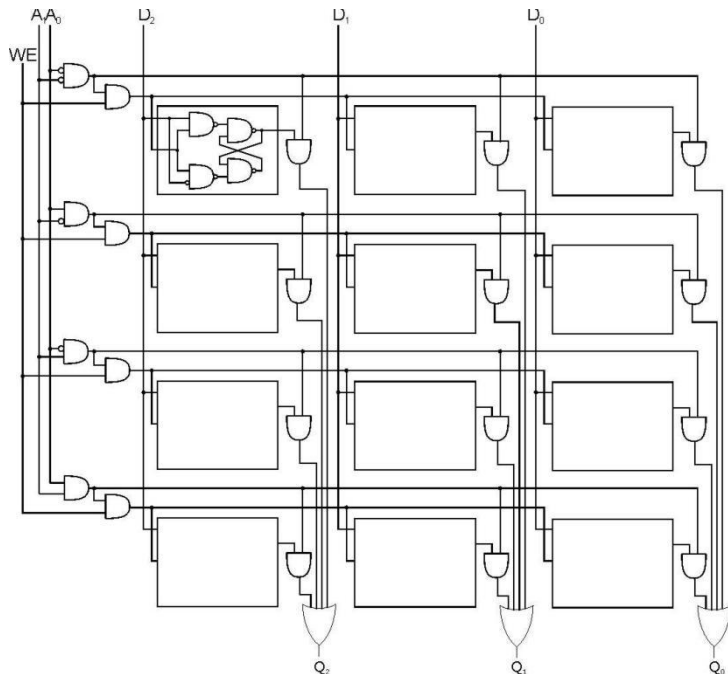
4×3 存储器：

结构： $\left. \begin{array}{l} 2-4 \text{ 译码器} \\ \text{写使能 WE} \end{array} \right\} \rightarrow \text{门控 D 锁存器} \rightarrow 4-1 \text{ 多路选择器} \rightarrow \text{门控 D 锁存器} \rightarrow 4-1 \text{ 多路选择器} \rightarrow \text{门控 D 锁存器} \rightarrow 4-1 \text{ 多路选择器}$

有 2^2 个存储单元，3 位寻址能力。

2 根地址线[1:0]：译码器

3 根数据线[2:0]：门控 D 锁存器。



24.字段：位组合的子单元。

25.地址：和每个单元联系在一起的唯一标识符。

26.地址空间：唯一可被识别的单元总数。

n 位地址，则地址空间为 2^n 。

27.寻址能力：存储在每个单元中的位数。

大多数存储器字节可寻址。

28.存储单元单位：1K= 2^{10} ，1M= 2^{20} ，1G= 2^{30} ，1T= 2^{40}

30.单元组 (nibble)：4bit

31.4GB 存储容量表示有 2^{32} 个存储单元，字节可寻址。

32.译码器某一位地址输出的 1 是被寻址的字线。

33.SRAM (静态随机访问存储器)

静态：只要供电，内部数据就不会丢失。

随机：任意顺序访问而不关心前次访问的单元。

34.时序逻辑电路既能存储信息，又能处理信息。输出值由当前输入和存储元件中的值共同决定。

35.状态：某一特定时刻，系统内所有相关部分的一个瞬态图。

36.状态数目必须有限，原因是存储元件容量有限。

37.有限状态机组成元素

- ①有限数目的状态。
- ②有限数目的外部输入。
- ③有限数目的外部输出。
- ④明确定义的状态转换函数。
- ⑤明确定义的外部输出函数。

38.状态图中圆对应一个状态，弧线与箭头确定一个状态的转换。

39.时钟电路：触发状态从一个向下一个转换的机制。

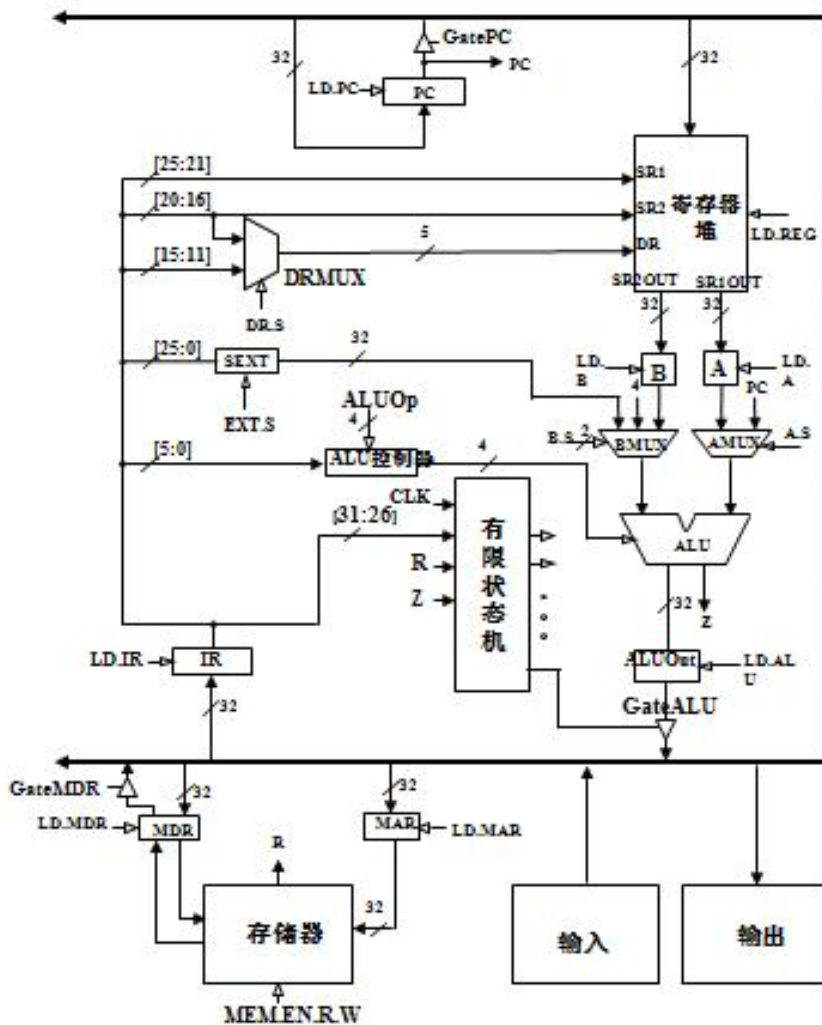
40.时钟周期：重复的时间间隔序列中的一个时间间隔。

41.存储元件使用主从触发器，即 2 个门控 D 锁存器。

第八章 冯·诺依曼模型

1.数据通路：计算机内部用于处理信息的所有元件总和。

2.DLX 子集数据通路



3.PC: 程序计数器（指令指针），32 位，存储下一条指令所在地址的寄存器。

4.IR: 指令寄存器，32 位。保存正在处理的指令。

5.多路选择器 {
DRMUX: 提供 5 位地址给寄存器堆。
AMUX
BMUX} 分别提供 32 位数值给 ALU。

6.DLX 数据通路采用总线结构，多时钟周期实现方案。

7.总线: 存储器和处理器，处理器和 I/O 设备间的通信。用黑色箭头框表示。

优点: 功能多成本低。

缺点: 产生通信瓶颈（总线上一次只传输一个值）。

8.三态设备（三角形）: 使计算机的控制逻辑一次只允许一个提供者向总线提供信息。位于每一个提供数据给总线的组件的输入箭头处。

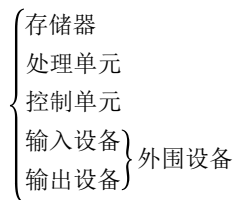
9.LD.X（加载使能）: 使信息允许加载时为 1，不允许加载时为 0。

10.Gate.X: 使数据与总线相连。

11.实心箭头: 通信流动数据元素。

12.空心箭头: 控制数据元素处理的控制信号。

13.冯诺依曼模型主要思想: 把程序和数据都作为一个二进制序列存储在存储器里，在控制单元的引导下一次执行一条指令。



14.由指令组成的程序和程序所需的数据位于存储器中。

15.MAR（地址寄存器）：32 位（DLX 有 2^{32} 个存储单元）。

16.MDR（数据寄存器）：32 位，存储连续 4 个单元的数据，或单元数据符号扩展的结果。

17.处理单元执行信息的实际处理。DLX 只包含 ALU 和寄存器堆。

18.ALU：算数逻辑单元。

19.字长：ALU 正常处理的信息量大小。ALU 有 32 位字长。

20.字：信息中的每一个元素。1 个字=4 个字节，32 位。

21.寄存器堆：靠近 ALU 提供少量存储空间用于临时存取数据的一组寄存器。

避免对存储器不必要的长时间访问。

每个寄存器包含一个字。

共 32 个通用寄存器（R0, R1, ..., R31），32 个浮点寄存器（F0, F1, ..., F31）。

22.控制单元指挥信息的处理。

{ 执行程序时，跟踪存储器中的指令。
处理指令时，跟踪指令的处理阶段。

23.指令包括操作码和操作数。

24.寻址模式：计算机将要读取/存储的存储单元的地址的机制。

25.多周期实现方案

①取指令

PC 中的内容加载到 MAR，PC 与 4 在 ALU 中做加法运算，并用结果改变 PC 的值。

查询存储器，把指令放进 MDR。

把 MDR 的内容加载入 IR。

②译码：识别指令而确定下一步。

取寄存器：为后面的阶段进行获取操作数的操作。（有的操作结果后面不会用到，但不浪费时间，原因：操作同时进行）

③执行：对上一阶段得到的寄存器的值执行算数/逻辑运算。

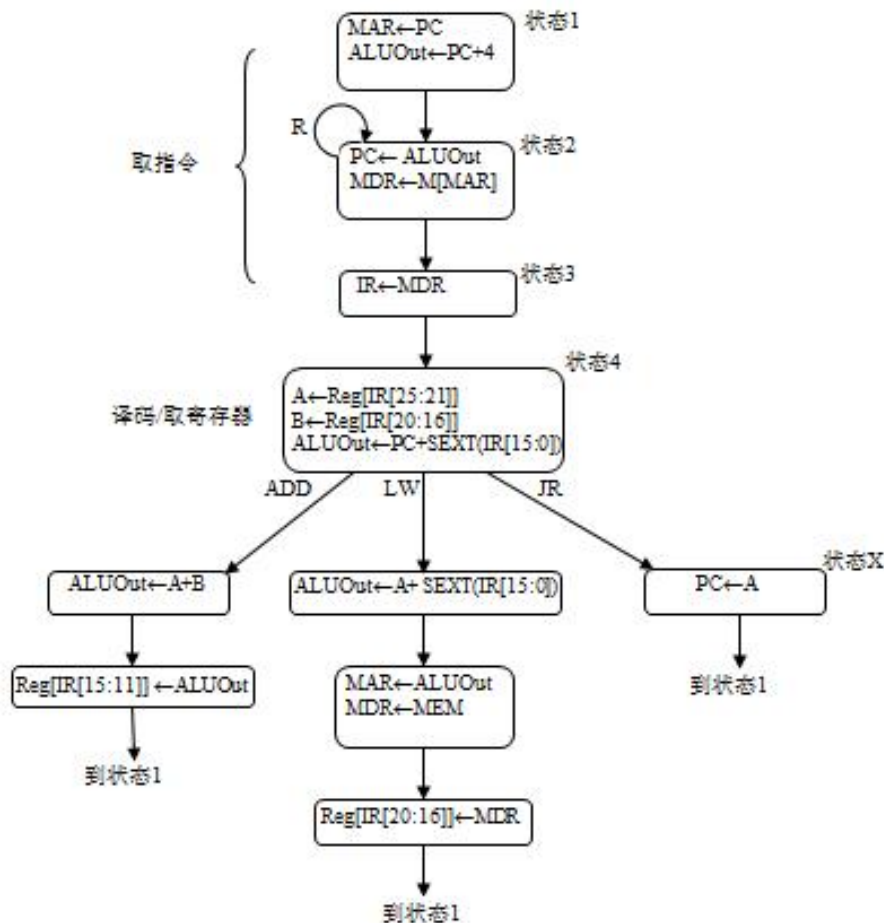
有效地址：计算出处理指令所需存储单元的地址。

完成分支：完成分支跳转。

④访问内存：获取内存中的数据。

⑤存储结果：将结果写入指定目标。

26.DLX 状态图（需要多个时钟周期，一个状态内的操作同时执行）



状态①

MAR ← PC

有限状态机使 Gate.PC 和 LD.MAR 设为 1，PC 与总线相连，当前时钟周期结束时将总线内容写入 MAR。

ALUOut ← PC + 4

有限状态机使 A.S (AMUX 选择信号) 设为 1，选择来自 PC 的输入；

有限状态机使 B.S (BMUX 选择信号) 设为 01，选择输入 4；

有限状态机使 ALUOp 设为 0001，在 ALU 中执行加法；

有限状态机使 LD.ALU 设为 1，结果存储与 ALUOut 中。

状态②

PC ← ALUOut

有限状态机使 Gate.ALU 和 LD.PC 设为 1，将 ALUOut 的值写入 PC。

MDR ← M[MAR]

有限状态机使 MEM.EN.R.W (写使能) 设为 0，LD.MDR 设为 1，读存储器。

R

读存储器需要多个时钟周期，读取时就绪信号 R 设为 0，读取结束时设为 1。

状态③

IR ← MDR

R 为 1 时，有限状态机使 Gate.MDR 和 LD.IR 设为 1，当前时钟周期结束时，IR 被写入。

状态④

A ← Reg[IR[25:21]], B ← Reg[IR[20:16]]

有限状态机使 LD.A 和 LD.B 设为 1，从 IR[25:21] 和 IR[20:16] 分别获得源操作数传给 ALU 的 A、B 寄存器。

ALUOut ← PC + SEXT(IR[15:0])

有限状态机使 A.S 设为 1，选择来自 PC 的输入；

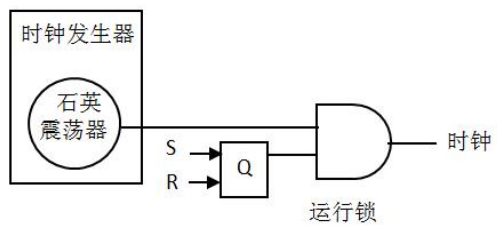
有限状态机使 EXT.S 设为 0，B.S 设为 00，选择 IR[15:0] 符号扩展结果；

有限状态机使 ALUOp 设为 0001，在 ALU 中执行加法；

有限状态机使 LD.ALU 设为 1，结果存储于 ALUOut 中。

27.时钟电路

Q=1	时钟电路输出与时钟发生器输出一样
Q=0（清0）	时钟电路输出为0，即停止指令运行。（HALT）



第九章 指令集结构

1. **ISA（指令集结构）** 指明了在一台机器上编写软件时所需要的全部信息。
2. **DLX 存储器组织** 有 2^{32} 个地址空间单元，8 位寻址能力（字节可寻址）。
3. **存储顺序：高位优先**（字的高位字节放在内存低地址端，低位字节放在高地址端）。
4. **边界对齐：字的起始地址必须是 4 的倍数（二进制末尾为 00）。**
5. 单精度浮点数要 1 个浮点寄存器，双精度浮点数要 2 个浮点寄存器。
6. 指令集 $\left\{ \begin{array}{l} \text{CISC(复杂指令集计算机): 开发容易, 执行效率低。} \\ \text{RISC(精简指令集计算机): 开发不足, 执行效率高。(DLX)} \end{array} \right.$
7. **DLX 数据类型** 支持二进制补码整数（8 位，16 位，32 位），单、双精度浮点数。
8. **DLX 指令可定义 127 条（操作码和函数），但 DLXISA 只定义了 91 条，未定义的被保留。**
9. **I-类型（立即数操作）**：[31:26]操作码，[25:21]源寄存器，[20:16]目标寄存器，[15:0]16 位立即数。
10. **R-类型（寄存器操作）**：[31:26]000000，[25:21]源寄存器 1，[20:16]源寄存器 2，[15:11]目标寄存器，[10:6]00000（未用），[5:0]函数。
11. **J-类型（跳转操作）**：[31:26]操作码，[25:0]26 位地址偏移量。
12. 指令类型
 - ① 算术/逻辑运算：处理整数。
 - ② 数据传送：在存储器和寄存器、寄存器和 I/O 设备间传送数据。
 - ③ 控制：改变指令执行顺序。
 - ④ 浮点：处理浮点数。
13. **DLX 只支持“基址+偏移量”寻址模式。**
14. 同一条指令，同一个寄存器可以同时作为源寄存器和目标寄存器。
15. 001011 SR SR 1111 1111 1111 1111 (xori sr,sr,xFFFF) 对 SR 按位取反。
16. 逻辑运算时，结果为真设目标寄存器值为 1，结果为假设目标寄存器值为 0。
17. **srai 按位算术右移，左侧空位符号扩展。每右移一次表示除以 2 一次。**
18. **srli 按位逻辑右移，左侧空位补 0。**
19. **slli 按位左移，右侧空位补 0。每左移一次表示乘以 2 一次。**
20. **lhi 加载高位立即数，将 imm16 左移 16 位，结果保存于 DR 中。与 addi 指令一起使用将较大的立即数赋给寄存器。**
21. **加载**：将数据从存储器移动到寄存器的过程。
22. **存储**：将数据从寄存器移动到存储器的过程。
23. **lb 和 sb**：加载和存储 8 位字节，一个存储单元和一个寄存器之间传送数据。
24. **lw 和 sw**：加载和存储 32 位子，四个连续的存储单元和一个寄存器之间传送数据。
25. **lb, sb, lw, sw** 将 16 位偏移量符号扩展至 32 位，与基址寄存器相加，获得存储器起始地址。
26. **lw 和 sw 计算所得的存储器起始地址必须是 4 的倍数。**
27. **绝对地址**：当基址寄存器为 r0（或值为 0）时，Imm16 就是访问存储器的地址。
28. 控制指令 $\left\{ \begin{array}{l} \text{条件分支} \\ \text{无条件跳转} \\ \text{子例程（函数）} \\ \text{TRAP} \\ \text{异常/中断返回} \end{array} \right.$
29. **beqz 条件假跳转**，若 SR1 值为 0，则从 PC+SEXT[Imm16]+4 开始执行；若 SR1 值不为 0，则 PC+4 开始执行。
30. **bnez 条件真跳转**，若 SR1 值不为 0，则从 PC+SEXT[Imm16]+4 开始执行；若 SR1 值为 0，则 PC+4 开始执行。
31. **地址限制**：只能跳转至立即数足够表示的范围内（当前 $2^{15}+3 \sim -2^{15}+4$ 的单元）。
32. 在 beqz 中使 SR1=R0，成为无条件分支。
33. **jr 无条件跳转**，跳转至 SR1 值作为地址的指令处。
34. **j 无条件跳转**，跳转至 PC+SEXT[PCOffset26]+4 处。
35. 无条件跳转比条件分支范围更广。
36. **trap 指令**，调用一个操作系统服务例程，[25:0]是 TRAP 向量。
37. C 语言修饰符 **long** 和 **short** 可以扩大或缩小数据类型位数，**仅当特定系统支持才有效。**
39. 类型提升的本质只能从较短的类型转换成较长的类型。

第十章 机器语言程序设计

1.顺序结构

2.条件结构

生成条件指令→条件分支指令→子任务 2→J 指令→子任务 1

条件分支指令偏移量为（子任务 2 指令数+1）×4。

J 指令偏移量为子任务 1 指令数×4。

3.重复结构

生成条件指令→条件分支指令→子任务→J 指令

条件分支指令偏移量为（子任务指令数+1）×4。

J 指令可以指向生成条件指令或条件分支指令。

4.设置值：不考虑之前运行的其他部分而直接设置数据。

5.运行：在 TRAPx00 或断点处停止。

6.按步运行：运行一定数量的指令后停止。

7.设置断点：在特定指令下停止。

8.Step Over：单步执行但将子例程当做一步来执行。

9.Step In：进入子例程并在子例程内部单步执行。

10.Step Out：已进入子例程时一步完成子例程。

11.输入数字字符时保存了其 ASCII 码，必须先转换再计算，数值=ASCII 码—x30。

12.编写子例程时若还调用了其他例程，应当先保存 r31。

13.子例程占用寄存器时需要保存并恢复。

第十一章 汇编语言

1.符号地址：使用符号表示存储单元的地址。

2.高级语言编程对用户更加友好，但放弃了对指令的精确控制。

3.程序执行前将汇编语言翻译成适合于期望执行计算机的 ISA 程序。翻译程序是**汇编器**，翻译过程是**汇编**。

4.指令不区分大小写。

5.自由格式，单词间和行间空格不改变程序意义。

6.寄存器共 32 个，R0、R1...R31。

7.寄存器限制

①输入输出：R4

②局部变量：R16、R17、R18、R19、R20、R21、R22、R23

③临时值：R8、R9、R10、R11、R12、R13、R14、R15、R24、R25

④全局指针：R28

⑤栈指针：R29

⑥帧指针（框架指针）：R30

8.立即数基符号：# 十进制；x 十六进制；b 二进制

9.算数/逻辑运算指令

I-类型：OPCODE DR SR1 Imm/LABEL

R-类型：OPCODE DR SR1 SR2

LHI 指令：LHI DR Imm/LABEL

10.数据传送指令

加载指令：LW/LB DR Imm/LABEL(SR1)

存储指令：SW/SB Imm/LABEL(SR1) DR

LW 和 SW 指令基址寄存器+偏移量得到的地址必须是 4 的倍数。

11.控制指令

条件分支指令：BEQZ/BNEZ SR1 LABEL

J 指令：J LABEL

JR 指令：JR SR1

TRAP 指令：TRAP Imm（trap 向量）

12.标记（LABEL）：用来明确标识存储单元的符号名。

13.标记由字母、数字、下划线组成，以字母、下划线、\$开头，以冒号：结尾。

14.指令操作码不可以作为标记名。

15.注释以分号；开头，至行末的内容被忽略。

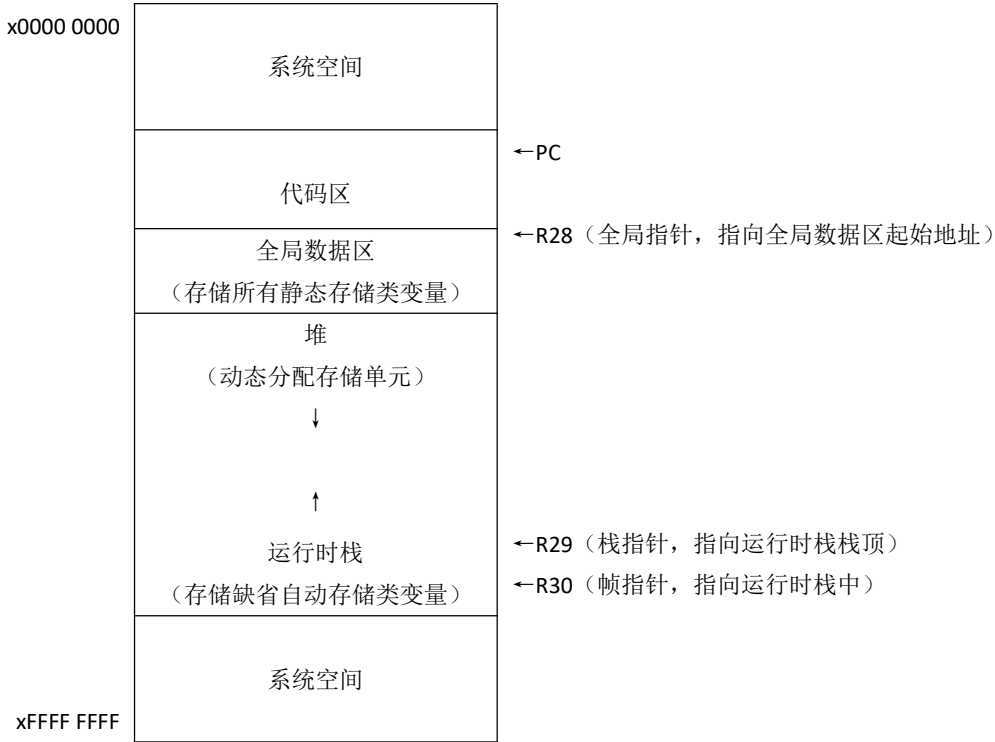
16.使用空格将程序对齐。

17.注释目的：提高可读性。

18.伪操作有助于汇编器把输入的字符串作为使用 DLX 汇编语言写的计算机程序，并将其翻译为 DLX 的 ISA 程序。汇编完成即被抛弃，程序执行过程中的伪操作不执行。

19.伪操作以点.开头。**第一行必须为.data address。必须有.global main**

- 20.汇编过程至少扫描两趟。
- 第一趟：构建符号表。
- 第二趟：把汇编语言指令翻译成机器语言指令。
- 21.LC：地址计数器
- 22.一条汇编语言指令可能翻译成多条机器语言指令。（大立即数由 LHI 实现）
- 23.可执行映像：当计算机开始一个程序的执行时的被执行实体。
- 24.链接过程由链接器完成。链接器为每个模块重新分配存储空间，因此.text address 和.data address 中的 address 是可省略的。
- 25.栈是一种抽象数据类型。
- 26.栈协议：后进先出（LIFO）
- 27.栈顶（TOP）：数据只能经栈顶处访问。
- 28.硬件栈：push 时数据向上移动，pop 时数据向下移动，栈顶固定在底层。
- 29.栈指针（R29）：存储栈顶的单元地址。push 时数据向上添加，栈顶上移一格，R2 减 4；pop 时数据不进行物理移动，栈顶下移一格，R29 加 4，原数据不删除。
- 30.push（压栈）：把一个元素插入栈。
- 31.push: subi r29,r29,#4
 sw 0(r29),SR
- 32.pop（出栈）：从栈中移出一个元素。
- 33.pop: lw DR,0(r29)
 addi r29,r29,#4
- 34.当变量个数多于寄存器数目，需要使用存储器。存储时边界对齐。
- 35.DLX 存储器组织



第十二章 输入和输出

1.I/O 设备至少包含两个设备寄存器。

- ①保存计算机和设备之间进行传输的数据。
- ②保存设备的状态信息。

2.访问 I/O 设备寄存器的两种机制。

- ①I/O 指令。(Intel x86 指令集, 使用 IN 和 OUT 指令)
- ②用于在通用寄存器和存储器之间传送数据的数据传送指令。(DLX)

3.内存映射: 每一个 I/O 设备寄存器都被分配一个存储器地址空间中的地址。

4.DLX 设备寄存器 (每个 I/O 寄存器映射 4 个存储器存储单元)

xFFFF 0000: 键盘状态存储器 (KBSR)

xFFFF 0004: 键盘数据寄存器 (KBDR)

xFFFF 0008: 显示器状态寄存器 (DSR)

xFFFF 000C: 显示器数据寄存器 (DDR)

xFFFF 00F8: 机器控制寄存器 (MCR)

5.被映射的存储单元不能作为存储单元使用, 加载数据时直接获得 I/O 数据寄存器中的数据。

6.异步: I/O 设备与微处理器不一致。

7.处理异步问题的协议或握手机制: 用 1 位状态寄存器标识就绪位。

8.打印机状态寄存器需要 2 位表示打印是否完成和打印机是否出错。

9.轮询: 通过处理器周期性检查状态位来判断是否执行 I/O 操作。

由处理器完全控制和执行通信工作。

缺点: 浪费大量处理时间。

10.中断驱动的 I/O: 处理器做自己的工作直到被信号打断。

由 I/O 设备控制交互。

11.KBDR 和 DDR[7:0]存放数据, [31:8]都为 0; KBSR 和 DSR[0]存放就绪位, [31:1]另有用途。

12.DLX 基本输入服务例程 In (轮询):

01		.data	x00003000	
02	SaveR1:	.space	4	;保存寄存器的存储单元
03	SaveR2:	.space	4	
04	SaveR3:	.space	4	
05	SaveR5:	.space	4	
06	KBSR:	.word	xFFFF0000	
07	KBDR:	.word	xFFFF0004	
08	DSR:	.word	xFFFF0008	
09	DDR:	.word	xFFFF000C	
0A	Newline:	.byte	x0A	;换行的 ASCII 码
0B	Prompt:	.ascii	" Input a character>"	;提示符字符串
0C	;保存例程使用的寄存器			
0D		.text	x00003100	
0E		sw	SaveR1(r0),r1	;保存例程所需寄存器
0F		sw	SaveR2(r0),r2	
10		sw	SaveR3(r0),r3	
11		sw	SaveR5(r0),r5	
12		lb	r2,Newline(r0)	
13		lw	r5,DSR(r0)	;测试输出寄存器是否就绪
14	L1:	lw	r3,0(r5)	
15		andi	r3,r3,#1	
16		beqz	r3,L1	;循环直到显示器就绪
17		lw	r5,DDR(r0)	

18		sw	0(r5),r2	;光标移到新行
19	;输出提示符			
1A		addi	r1,r0,Prompt	;提示符字符串起始地址
1B	LOOP:	lb	r2,0(r1)	;输出提示符
1C		beqz	r2,Input	;提示符字符串结束
1D		lw	r5,DSR(r0)	
1E	L2:	lw	r3,0(r5)	
1F		andi	r3,r3,#1	
20		beqz	r3,L2	;循环直到显示器就绪
21		lw	r5,DDR(r0)	
22		sw	0(r5),r2	;输出下一个提示符
23		addi	r1,r1,#1	;提示符指针加 1
24		j	LOOP	;获取下一个提示符
25	;输入回显			
26	Input:	lw	r5,KBSR(r0)	
27	L3:	lw	r3,0(r5)	
28		andi	r3,r3,#1	
29		beqz	r3,L3	;轮询直到一个字符被键入
2A		lw	r5,KBDR(r0)	
2B		lw	r4,0(r5)	;将输入的字符加载到 R4
2C		lw	r5,DSR(r0)	
2D	L4:	lw	r3,0(r5)	
2E		andi	r3,r3,#1	
2F		beqz	r3,L4	;循环直到显示器就绪
30		lw	r5,DDR(r0)	
31		sw	0(r5),r4	;将输入字符回显
32	;输出新行			
33		lb	r2,Newline(r0)	
34		lw	r5,DSR(r0)	
35	L5:	lw	r3,0(r5)	
36		andi	r3,r3,#1	
37		beqz	r3,L5	;循环直到显示器就绪
38		lw	r5,DDR(r0)	
39		sw	0(r5),r2	;移动光标到新行
3A	;恢复例程使用的寄存器			
3B		lw	r1,SaveR1(r0)	;将寄存器恢复为原先的值
3C		lw	r2,SaveR2(r0)	
3D		lw	r3,SaveR3(r0)	
3E		lw	r5,SaveR5(r0)	
3F		jr	r31	;从 TRAP 返回

13.DLX 基本输出服务例程 OUT (轮询):

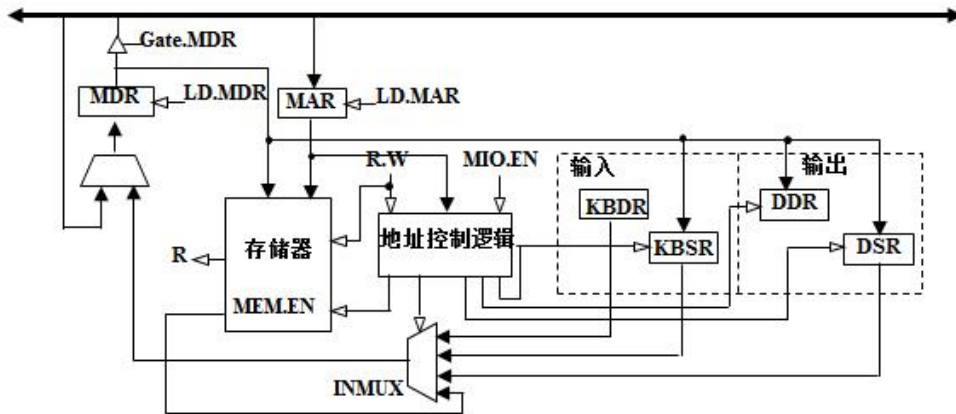
01		.data	x00002800	
02	SaveR1:	.space	4	;保存寄存器的存储单元
03	SaveR2:	.space	4	
04	DSR:	.word	xFFFF0008	
05	DDR:	.word	xFFFF000C	
06	;			
07		.text	x00002900	
08		sw	SaveR1(r0),r1	;保存例程所需寄存器

09	sw	SaveR2(r0),r2	
0A	lw	r1,DSR(r0)	
0B	START: lw	r2,0(r1)	;测试输出寄存器是否就绪
0C	andi	r2,r2,#1	
0D	beqz	r2,START	
0E	lw	r1,DDR(r0)	
0F	sw	0(r1),r4	
10	lw	r1,SaveR1(r0)	;将寄存器恢复为原先的值
11	lw	r2,SaveR2(r0)	
12	jr	r31	;从 TRAP 返回

14. 内存映射 I/O 数据通路

地址控制逻辑控制输入输出操作。

3 个块输入 { **MIO.EN**: 表示当前时钟周期里是否发生存储器或 I/O 数据传送操作。
MAR: 存储单元地址或 I/O 设备寄存器内存映射地址。
R.W: 表示加载或存储。



第十三章 自陷例程和中断

- 1.硬件寄存器有特权，不拥有适当特权级别的程序不能访问。
- 2.输入输出时使用 TRAP 指令使操作系统完成，用户程序不必知道实现细节，其他用户程序被保护，避免用户程序员不恰当行为的后果。
- 3.服务调用或系统调用：用户程序请求操作系统完成任务。
- 4.DLX 有 256 个服务例程，用 TRAP 机制调用。

x06	GETC	从键盘读取一个字符，将其 ASCII 码复制到 R4[7:0]
x07	OUT	将 R4[7:0]中的字符输出到显示器
x08	PUTS	将 R4 所指的地址开头的一个字符串输出到显示器，每个字符占用一个存储单元，字符串以 x00 终止
x09	IN	输出 “Input a character>” 到显示器，从键盘读取一个字符后回显到显示器上，并将其 ASCII 码复制到 R4[7:0]
x0A	GETS	两个参数 R4（字符串起始地址）和 R5（长度 n）,从键盘读取 n-1 个字符，如果输入小于 n-1，则至回车结束，读入缓冲区，并在末尾加上 x00
X00	HALT	输出 “Halting the machine.”，并停止程序

5.TRAP 向量表或系统控制块

- ①存储在 x00000000 到 x000003FF 中。
- ②包括 256 个服务例程的起始地址，每个地址占用 4 个存储单元，每个服务例程占用 2^{10} 个存储单元（数据区占 x100，服务例程占 x300，共 x400）。
- ③每个服务例程的数据段起始地址均为其代码段起始地址前 x100 个单元的位置。
- ④部分 TRAP 向量表

TRAP 向量	向量表地址	服务例程地址	数据区地址
x06 GETC	x0000 0018	x0000 2500	x0000 2400
x07 OUT	x0000 001C	x0000 2900	x0000 2800
x08 PUTS	x0000 0020	x0000 2D00	x0000 2C00
x09 IN	x0000 0024	x0000 3100	x0000 3000
x0A GETS	x0000 0028	x0000 3500	x0000 3400
x00 HALT	x0000 0000	xFFFE 0100	xFFFE 0000

6.TRAP 指令执行步骤

- ①26 位 TRAP 向量符号扩展到 32 位，再左移 2 位（乘以 4）形成地址，该地址加载到 MAR 中。
 - ②加载到 MAR 中的地址中的记录被读取，并将其加载到 MDR 中。
 - ③通用寄存器 R31 被加载为 PC 当前内容（已经被加 4）。
 - ④MDR 内容被加载到 PC 中，并完成指令的执行。
 - ⑤返回用户程序。
- 7.返回用户程序时使用指令 JR R31，用助记符 RET 表示，调用 TRAP 指令后 R31 中的数据被破坏。
- 8.MCR（机器控制寄存器）的[0]位存储运行锁。

9.HALT 服务例程

01		.data	xFFFE0000	
02	SaveR1:	.space	4	;保存寄存器的存储单元
03	SaveR2:	.space	4	
04	SaveR4:	.space	4	
05	SaveR31:	.space	4	
06	Newline:	.byte	x0A	
07	Message:	.ascii	“Halting the machine.”	
08		.align	2	
09	MCR:	.word	xFFFF00F8	
0A	;			
0B		.text	xFFFE0100	

```

0C          sw      SaveR1(r0),r1      ;保存例程所需寄存器
0D          sw      SaveR2(r0),r1
0E          sw      SaveR4(r0),r1
0F          sw      SaveR31(r0),r31
10      ;
11      ;输出停机消息
12      ;
13          lb      r4,Newline(r0)
14          trap    x07
15          addi    r4,r0,Message
16          trap    x08
17          lb      r4,Newline(r0)
18          trap    x07
19      ;
1A      ;清空 xFFFF 00F8 的 0 位， 停机
1B      ;
1C          lw      r1,MCR(r0)
1D          lw      r2,0(r1)          ;加载 MCR 值到 R2 中
1E          andi    r2,r2,#-2          ;清空 MCR[0]
1F          sw      0(r1),r2          ;将 R2 值存储到 MCR 中
20      ;至此已经停机，若未重置以下指令不会执行
21      ;从 HALT 例程返回
22      ;
23          lw      r1,SaveR1(r0)      ;将寄存器恢复为原先的值
24          lw      r2,SaveR2(r0)
25          lw      r4,SaveR4(r0)
26          lw      r31,SaveR31(r0)
27          jr      r31                ;从 TRAP 返回

```

10.**HALT** 指令使用了 **R1、R2、R4、R31** 且停机时数据被破坏。

11.PUTS 服务例程

```

01          .data    x00002C00
02      SaveR2: .space 4          ;保存寄存器的存储单元
03      SaveR3: .space 4
04      SaveR4: .space 4
05      SaveR5: .space 4
06      DSR:      .word xFFFF0008
07      DDR:      .word xFFFF000C
08      ;
09      ;
0A          .text    x00002D00
0B          sw      SaveR2(r0),r2      ;保存此例程所需寄存器
0C          sw      SaveR3(r0),r3
0D          sw      SaveR4(r0),r4
0E          sw      SaveR5(r0),r5
0F      ;
10      ;对字符串中的每一个字符进行循环
11      ;
12      LOOP:  lb      r2,0(r4)          ;取得字符
13          beqz    r2,Return          ;如果是 0，字符串结束

```

```

14          lw      r5,DSR(r0)
15  L2:      lw      r3,0(r5)      ;测试输出寄存器是否就绪
16          andi    r3,r3,#1
17          beqz    r3,L2          ;循环直到显示器就绪
18          lw      r5,DDR(r0)    ;
19          sw      0(r5),r2      ;显示字符
1A          addi    r4,r4,#1      ;指针加 1
1B          j      LOOP          ;获取下一个字符
1C          ;
1D          ;从服务调用请求返回
1E  Return:  lw      r2,SaveR2(r0) ;将寄存器恢复为原先的值
1F          lw      r3,SaveR3(r0)
20          lw      r4,SaveR4(r0)
21          lw      r5,SaveR5(r0)
22          jr      r31          ;从 TRAP 返回

```

12.caller-save（调用者保存）：调用程序保存占用的寄存器。

13.callee-save（被调用者保存）：被调用程序保存占用的寄存器。

14.中断驱动的 I/O 的本质是 I/O 设备能够

- ①强制程序停止。
- ②让处理器执行 I/O 设备的请求。
- ③让停止的程序继续执行。

15.某个 I/O 设备能够生成中断请求，必须具备

- ①I/O 设备需要服务。
- ②设备有权请求服务。

16.IE（中断允许位）在设备状态寄存器（KBSR 和 DSR）的[1]位，1 表示允许中断，0 表示不允许中断。

17.中断请求信号（IRQ）是 IE 位与就绪位逻辑与运算的结果。

18.原因寄存器（CAUSE）记录哪些设备发出中断信号，只有特权模式（操作系统）下才能访问。

[15:8]为中断未决位，[15:10]为硬件中断未决位，[9:8]为软件中断未决位。

CAUSE[11]代表键盘中断未决位，CAUSE[10]代表显示器中断未决位。

按照优先级高低从左至右排列。

未决中断等到相应 SR 中断屏蔽位为 1 时才能引起处理器处理。

19.SR（状态寄存器）

[0]位可以改写所有设备的 IE 位，只有特权模式（操作系统）下才能访问。

[1]位表示正在运行的程序处于特权（管理员或内核）模式时为 0，处于用户模式时为 1。

[2]位中断发生时保存 SR[0]的值。

[3]位中断发生时保存 SR[1]的值。

[15:8]位是中断屏蔽位（中断掩码位），给出中断阻塞方案，[15:10]为硬件中断屏蔽位，[9:8]为软件中断屏蔽位。优先级高低从左至右排列。1 表示允许中断，0 表示屏蔽中断。

20.程序状态：程序影响的所有资源所包含的内容的瞬态图。

21.EPC：保存中断发生时 PC 的值，只有特权模式（操作系统）下才能访问。

22.进入中断服务例程时，应屏蔽所有中断，原因：避免受到来自其他设备的中断信号干扰。

23.优先级：执行的紧急程度。请求的优先级必须高于希望中断的程序。

24.DLX 硬件优先级从低到高分 PL0、PL1、PL2、PL3、PL4、PL5，速度越高的 I/O 设备优先级越高。

25.键盘优先级为 1，显示器优先级为 0。

26.中断服务例程

```

01          .data      x80000000
02  SaveR1:  .space    4          ;保存寄存器的存储单元
03  SaveR2:  .space    4
04  SaveR3:  .space    4

```

```

05  SaveR5:    .space    4
06  SaveR6:    .space    4
07  SaveR7:    .space    4
08  KBDR:      .word     xFFFF0004
09  DDR:       .word     xFFFF000C
0A  ;
0B          .text      x80001000
0C  ;保存中断服务例程需要的寄存器
0D          sw         SaveR1(r0),r1
0E          sw         SaveR2(r0),r2
0F          sw         SaveR3(r0),r3
10          sw         SaveR5(r0),r5
11          sw         SaveR6(r0),r6
12          sw         SaveR7(r0),r7
13  ;是否有允许的中断
14          movs2i     r1,x0D          ;CAUSE
15          movs2i     r2,x0C          ;SR
16          movs2i     r6,x0E          ;EPC
17          andi       r3,r1,xFF00     ;CAUSE[15:8]
18          andi       r5,r2,xFF00     ;SR[15:8]
19          and        r3,r3,r5
1A          beqz       r3,DONE          ;没有允许的中断，返回
1B  ;按照优先级顺序，依次处理
1C  TEST5:     slli     r3,r3,#16
1D          slti      r5,r3,#0
1E          addi       r7,r0,TEST4
1F          bnez       r5,DEV5          ;优先级为 PL5 的设备
20  TEST4:     slli     r3,r3,#1
21          slti      r5,r3,#0
22          addi       r7,r0,TEST3
23          bnez       r5,DEV4          ;优先级为 PL4 的设备
24  TEST3:     slli     r3,r3,#1
25          slti      r5,r3,#0
26          addi       r7,r0,TEST2
27          bnez       r5,DEV3          ;优先级为 PL3 的设备
28  TEST2:     slli     r3,r3,#1
29          slti      r5,r3,#0
2A          addi       r7,r0,TEST1
2B          bnez       r5,DEV2          ;优先级为 PL2 的设备
2C  TEST1:     slli     r3,r3,#1
2D          slti      r5,r3,#0
2E          addi       r7,r0,TEST0
2F          bnez       r5,DEV1          ;优先级为 PL1 的键盘
30  TEST0:     slli     r3,r3,#1
31          slti      r5,r3,#0
32          bnez       r5,DEV0          ;优先级为 PL0 的显示器
33  ;...（软件中断测试）
34          j          DONE
35  ;处理中断

```

```

36    ;... (其它设备)
37    DEV1:    lw        r1,KBDR(r0)
38                lw        r4,0(r1)        ;输入字符加载到 R4
39                jr        r7
3A    DEV0:    lw        r1,DDR(r0)
3B                sw        0(r1),r4        ;R4 中的字符输出到显示器
3C    ;将寄存器恢复为原先的值
3D    DONE:    lw        r1,SaveR1(r0)
3E                lw        r2,SaveR2(r0)
3F                lw        r3,SaveR3(r0)
40                lw        r5,SaveR5(r0)
41                lw        r6,SaveR6(r0)
42                lw        r7,SaveR7(r0)
43                movi2s    r0,x0D        ;清空 CAUSE
44                rfe

```

27.中断嵌套：中断服务例程执行时允许被比其优先级高的设备中断。

- ①保存 SR 和 EPC。
- ②修改 SR[15:8]，屏蔽比该设备优先级低（或相等）的其他设备的中断，允许优先级高的设备中断。
- ③将 SR[0]设为 1，允许中断。
- ④结束中断前设 SR[0]为 0（恢复 SR 和 EPC 时不允许被中断）
- ⑤恢复 SR 和 EPC。

28.中断嵌套时保存程序状态使用栈。

29.C 语言程序输入输出通过 I/O 库函数执行。

30.所有基于字符的输入和输出都是对**流**执行的。

31.当一个字符被键入，它被添加到输入流的结尾处。读取键盘的输入，总是从输入流开头处读取。

32.程序要打印的 ASCII 码字符序列被添加到输出流结尾处，输出时总是从输出流开头处输出，**每次流中的一个字符使用后被消耗，未被使用的字符将保留在流中。**

33.使用流使输入和输出以其各自速率操作而不用等待另一个就绪。

34.stdin（**标准输入流**）缺省映射到键盘，stdout（**标准输出流**）缺省映射到显示器。

35.putchar()输出一个字符，getchar()输入一个字符（**从输入流中读取一个字符，当输入流为空时等待**）。二者必须包含 stdio.h 头文件。

36.I/O **流缓冲**：每个键盘上的输入都被底层操作系统软件捕获，并被保存在一个小的数组缓冲区里，**直到用户按下回车键，缓冲区才被释放到输入流中，且回车键本身也作为换行字符加入输入流。**

优点：使用户能使用退格键删除以便编辑输入的内容，并按下回车键确认其输入。

37.使用 printf 输出%使用序列“%%”。

38.空白字符：空格、水平制表符、新行、回车、垂直制表符、换页。

39.scanf 输入时**抛弃所有空白字符（但不会连接空白字符两边的内容）**，并依次匹配符合的格式，一旦遇到不符的格式即终止输入，以回车确认输入。此时**未匹配的变量未被赋值而保持初值。**

40.scanf 函数返回成功转换的参数个数，printf 函数返回输出的字符串字符数（转义序列和格式说明计一个，不包括‘\0’），putchar 函数和 getchar 函数分别返回输出和输入的字符 ASCII 码值。

第十四章 子例程

1. **子例程**：在程序内不必在每次需要时均说明原程序段全部细节，而是通过多次调用来实现的程序片段。
优点：可以由不同程序员分别实现需要程序片段的程序和程序片段。
2. **库**：提供的程序片段的集合。
3. **调用/返回机制**：实现子例程的机制。
 - ①调用机制计算子例程起始地址，加载到 PC，**保存返回地址于 R31 中**。
 - ②返回机制使用返回地址加载 PC。
4. TRAP 服务例程包括操作系统资源，需要访问计算机底层硬件的特权，由管理计算机资源的系统程序员编写。
5. JAL 和 JALR 在 R31 中保存子例程需要返回的地址，返回地址为当前 PC+4，并跳转至子例程。
6. JAL 链接：[31:26]101110，[25:0]26 位地址偏移量，子例程目标地址为 PC+4+SEXT[PCOffset26]。
7. JALR 链接：[31:26]101111，[25:21]源寄存器，[20:0]0 0000 0000 0000 0000（未用），PC 直接加载 SR1 中的地址。
8. 在子例程中以 jr r31 或 ret 结束子例程并返回。
9. **变元（arguments）**：传给子例程的值。
10. **返回值（return values）**：从子例程传出的值。
11. 库函数由编译器和操作系统设计者提供。
12. 库例程标记在库中被标记为 **.global**，用户程序调用时需要指出 **.extern**。

第十五章 函数

1. C 语言中子程序称为**函数**。
2. C 语言是面向函数的，C 程序本质上是函数的集合，每条语句属于并仅属于一个函数。
3. C 语言程序总是从 main 函数开始，在 main 函数中调用其他函数，这些函数也可以依次调用更多函数，控制最终会返回 main 函数，main 函数结束时程序结束。
4. **函数声明**称为**函数原型**，包括函数名称、返回值类型、输入值列表，以分号结束。
5. **没有返回值的函数返回值类型为 void**。
6. 函数名称按标识符命名规则。
7. **帕斯卡命名法**：单词首字母大写，其余部分小写。
8. **骆驼式命名法**：单词首字母大写，其余部分小写，但第一个单词首字母小写。
9. **下划线法**：函数名中的每一个逻辑断点都有一个下划线来标记。
10. 函数声明括号中描述函数需要输入的参数的类型和顺序，**可以不指明参数名而只写参数类型**，参数名也可以与函数定义时的形式参数不同。不需要参数时括号里为空。
10. 在调用者内部被传给被调用者的值被称为**变元**。
11. 函数定义括号中是**形式参数列表**，变元与形式参数类型和顺序匹配。
12. **任何调用者的局部变量对被调用函数不可见**。
13. 用 **return** 指明返回的值，必须与声明返回的类型匹配。
14. **若有返回值的函数没有使用 return 语句，则最后一条语句的值作为返回值返回给调用者**。
15. 函数定义在函数调用前，可以不使用函数声明。
16. **变元运算顺序自右向左**。例：function(i,i++);。
17. 一个函数能被任何一个函数调用（包括其本身）。
18. C 函数在 DLX 底层实现时，参数个数若多于 4 个将使用存储器，且使用运行时栈机制，这段存储空间称为函数的**栈框架或活动记录**。
19. 编译器为每一次函数调用，在存储器中分配一个活动记录；当函数返回时，它的活动记录将被回收，以便分配给后面的函数；每一次函数调用都会在存储器中为其局部数值获得它自己的空间。优点：允许函数递归。
20. R29 栈指针指向栈顶，R30 框架指针指向活动记录底。
21. **动态链接**：为调用者的框架指针制作的副本。
22. **叶函数**：在执行过程中没有调用其他函数的函数，且没有改变 R31。
23. 头文件包含函数声明，预处理宏，但不包含库函数的源代码，即头文件已编译。
24. 数学库函数使用头文件 math.h。
25. **printf 函数格式说明少于参数时，将使用期望在存储器指定位置的垃圾值**。
26. **scanf 函数返回值为该函数在输入流中成功扫描的格式说明的个数**。

27.将 C 语言函数调用翻译为 DLX 汇编语言

①通过栈指针 R29 对局部变量 R16-R23、临时寄存器 R8-R15、R24、R25、参数传递寄存器 R2、R3 压栈（只对将要修改但还需使用的寄存器压栈）。

```
subi r29  r29  #4
sw 0(r29)  SR1
subi r29  r29  #4
sw 0(r29)  SR2
.....
```

```
subi r29  r29  #4
sw 0(r29)  SRn
```

②参数由 R4-R7 传递。

③返回值由 R2 和 R3 传递。

④通过栈指针 R29 将压栈的寄存器出栈。

```
lw SRn  0(r29)
addi r29  r29  #4
.....
lw SR2  0(r29)
addi r29  r29  #4
lw SR1  0(r29)
addi r29  r29  #4
ret
```

第十六章 指针和数组

1.指针：一个存储对象的地址。

2.数组：存储器中被连续排列的一系列数据。

3.变元总是以值的形式从主调函数传递到被调用函数。

4.声明指针变量：类型 *变量名；。星号*前后空格可有可无，指针变量类型视为类型*。指向指针的指针使用多个星号*。

5.地址运算符&：生成它操作数的存储地址。

6.间接运算符*：间接操作存储对象里的值。

7.赋值运算符右侧的间接运算符*生成 lw 指令，取出存储的值；赋值运算符左侧的间接运算符*生成 sw 指令，将右值存储到存储单元中。

8.scanf 函数格式用字符串后的参数必须使用指针。

9.主调函数的局部变量可以在被调用函数中通过指针间接修改。

10.指针变量可以赋值 NULL，表示不指向任何变量的空指针。NULL 是特别定义的预处理宏，等于 0，因为没有有一个有效的存储对象可以存储在单元 0 中，所以空指针不指向任何变量。

11.声明数组：类型 数组名[数组大小]；。数组名[0]被分配到最低存储地址，数组名[n-1]被分配到最高存储地址。

12.访问数组通过基址+偏移量访问。数组的基址是数组名[0]的地址。

13.数组的大小使用预处理宏有时比较恰当。

14.格式%Nd 表示十进制补码整数占 N 位，如果不足 N 位，则左边补空格。

15.格式%-Nd 表示十进制补码整数占 N 位，如果不足 N 位，则右边补空格。

16.原因：不会在调用函数时把每个元素从一个活动记录复制到另一个活动记录中而花费大量时间。

17.一维数组的名字就是该数组的基址，可以直接使用但不可以被修改，类型是声明时的类型。例如：int x[10]；，则 x，&x[0]，&x 三者等价，类型都为 int*。

18.函数声明和函数定义时的参数若为一维数组，数组大小可以留空，形如“类型 数组名[]”；也可以使用指针，形如“类型 *数组名”。例如：void function(int x[])；或 void function(int *x)；。

19.函数引用时，参数若为数组必须使用地址（一般直接写数组名）。参数传递后被调函数对应的数组形式参数名被赋值为传递的数组参数基址。

```
例如：int main(){
        int x[10];
```

```

    function(x);
}
void function(int cox[]){
    cox[1]=0;
}

```

则 cox 被赋值为 x (&x[0])。

20.主调函数的数组会在被调用函数中被直接修改并可见，包括使用“形式参数数组”或者引用“基址+偏移量指针”。偏移量是数组号而不用考虑所占存储单元数。

例如：void function(int cox[]){
 cox[1]=0; 或 *(cox+1)=0;
}

21.字符串：表示文本的字符序列，字符数组。

22.字符串用双引号“ ”包括。

23.字符串赋初值时编译器在末尾自动加上 0 ('\0')，不足位会补 0。

24.以空结尾的字符串'\0' 占用一个数组元素。

25.格式序列%s 在 printf 函数中，打印以参数表示的地址开始的字符串，以'\0'结尾。

26.格式序列%s 在 scanf 函数中，在输入流中读入从第一个非空字符开始，到下一个空白字符之间的字符，并在末尾自动加上一个'\0'，存储于以参数表示的地址为首地址的存储器中。最后的空白字符以及其后的内容保留在输入流中。

27.scanf 函数不检查数组大小，当输入超过数组大小时，将强制修改数组后面单元中的内容。

28.字符串处理函数使用头文件 string.h。

29.C 语言不提供防止超出数组大小（或边界）的保护措施，访问超出数组大小的存储单元会修改其中的内容，因此需要防御性编程。

30.C89 规定编译程序必须知道数组大小，即声明时数组大小不能使用变量。

31.二维数组声明：类型 数组名[行数][列数]；。分配空间时先按行后按列分配。

32.二维数组名是该数组的基址，数组名[行数]是数组中该行的首地址。例如：char a[5][5]；，则 a，&a[0][0]，a[0]，&a，&a[0]五者等价；a[1]，&a[1][0]，&a[1]三者等价。

33.strcmp 函数比较两个字符串，原型为 int strcmp(char *string1,char *string2)；。若 string1 大于 string2，则返回 1；若 string1 小于 string2 则返回-1；若 string1 等于 string2，则返回 0。比较方法为 ASCII 码字典序。

34.strcpy 函数复制字符串，strcpy(char *string1,const char *string2)；。将 string2 复制到 string1 中。

35.函数声明和定义时若参数为二维数组，一定要注意参数使用指针或地址。

①若在被调函数内对整个数组操作，则参数是整个数组，可以省略行数，函数引用参数是数组基址。

例如：有二维数组 int x[5][4]，则 void function(int [5][4])或 void function(int[][4])或 void function(int (*x)[4])中可以对整个数组 x 操作，函数引用为 function(x)；。

②若在被调函数内对某行操作，则参数是指针。被调函数的形式参数名已经表示原数组的该行地址。

例如：有二维数组 int x[5][4]，若函数引用为 function(x[2])；，则 void function(int *x)中只能操作第三行，x[3]表示 x[2][3]。

DLX 指令集

					31	26	25	21	20	16	15	11	10	6	5	0
加	ADD	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	000001						
加	ADDI	DR,	SR1,	Imm/LABEL	000001	SR1	DR	Imm16								
减	SUB	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	000011						
减	SUBI	DR,	SR1,	Imm/LABEL	000011	SR1	DR	Imm16								
按位与	AND	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	001001						
按位与	ANDI	DR,	SR1,	Imm/LABEL	001001	SR1	DR	Imm16								
按位或	OR	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	001010						
按位或	ORI	DR,	SR1,	Imm/LABEL	001010	SR1	DR	Imm16								
按位异或	XOR	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	001011						
按位异或	XORI	DR,	SR1,	Imm/LABEL	001011	SR1	DR	Imm16								
加载高位	LHI	DR,	Imm		001100	00000	DR	Imm16								
左移	SLL	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	001101						
左移	SLLI	DR,	SR1,	Imm/LABEL	001101	SR1	DR	Imm16								
逻辑右移	SRL	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	001110						
逻辑右移	SRLI	DR,	SR1,	Imm/LABEL	001110	SR1	DR	Imm16								
算数右移	SRA	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	001111						
算数右移	SRAI	DR,	SR1,	Imm/LABEL	001111	SR1	DR	Imm16								
小于	SLT	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	010000						
小于	SLTI	DR,	SR1,	Imm/LABEL	010000	SR1	DR	Imm16								
小于等于	SLE	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	010010						
小于等于	SLEI	DR,	SR1,	Imm/LABEL	010010	SR1	DR	Imm16								
等于	SEQ	DR,	SR1,	SR2	000000	SR1	SR2	DR	00000	010100						
等于	SEQI	DR,	SR1,	Imm/LABEL	010100	SR1	DR	Imm16								
加载字节	LB	DR,	Imm/LABEL(SR1)		010110	SR1	DR	Imm16								
存储字节	SB	Imm/LABEL(SR1), DR			010111	SR1	DR	Imm16								
加载字	LW	DR,	Imm/LABEL(SR1)		011100	SR1	DR	Imm16								
存储字	SW	Imm/LABEL(SR1), DR			011101	SR1	DR	Imm16								
(授权)存储字	MOVI2S	GPR	NUMBER		100010	00000	GPR	NUMBER	000 0000 0000							
(授权)加载字	MOV2SI	GPR	NUMBER		100011	00000	GPR	NUMBER	000 0000 0000							
条件假跳转	BEQZ	SR1,	LABEL		101000	SR1	00000	Imm16								
条件真跳转	BNEZ	SR1,	LABEL		101001	SR1	00000	Imm16								
无条件跳转	J	LABEL			101100	PCOffset26										
无条件跳转	JR	SR1			101101	SR1	0 0000 0000 0000 0000 0000									
从 TRAP 返回	RET	(JR R31)			101101	11111	0 0000 0000 0000 0000 0000									
链接	JAL	LABEL			101110	PCOffset26										
链接	JALR	SR1			101111	SR1	0 0000 0000 0000 0000 0000									
自陷 GETC	TRAP	x06			110000	00 0000 0000 0000 0000 0000 0110										
自陷 OUT	TRAP	x07			110000	00 0000 0000 0000 0000 0000 0111										
自陷 PUTS	TRAP	x08			110000	00 0000 0000 0000 0000 0000 1000										
自陷 IN	TRAP	x09			110000	00 0000 0000 0000 0000 0000 1001										
自陷 GETS	TRAP	x0A			110000	00 0000 0000 0000 0000 0000 1010										
自陷 HALT	TRAP	x00			110000	00 0000 0000 0000 0000 0000 0000										
(授权)恢复 PC	RFE				110001	00 0000 0000 0000 0000 0000 0000										

伪操作

.data address	DLX 数据存放在数据区地址 address。
.align n	将下面的数据或代码加载到二进制以 n 个 0 结尾的地址中。
.space size	在数据区留出 size 个连续存储单元。
.text address	DLX 指令存放在地址 address 中。
.global label	使标记 label 全局（使以后加载的其他文件也可以使用此标记）。 必须有.global main，且执行从标记 main 的指令开始。
.extern label	使用其他文件的全局标记 label。
.byte byte1,byte2 ...	将字节 1、字节 2、...连续存储到存储单元中。
.word word1,word2 ...	将字 1、字 2、...连续存储到存储单元中。
.ascii “string1”, “string2” ...	将字符串 1、字符串 2、...连续存储到存储单元中。
.asciiz “string1”, “string2” ...	分别在在字符串 1、字符串 2、...末尾添加 x00 后连续存储到存储单元中。

设备寄存器映射地址

MCR	机器控制寄存器	xFFF 00F8
KBSR	键盘状态寄存器	xFFF 0000
KBDR	键盘数据寄存器	xFFF 0004
DSR	显示器状态寄存器	xFFF 0008
DDR	显示器数据寄存器	xFFF 000C

特殊寄存器编号

SR	12	x0C	中断屏蔽，中断允许
CAUSE	13	x0D	未决中断位
EPC	14	x0E	PC

DLX 寄存器

GPR 通用寄存器

I/O 寄存器：R4

局部变量寄存器：R16、R17、R18、R19、R20、R21、R22、R23

临时寄存器：R8、R9、R10、R11、R12、R13、R14、R15、R24、R25

全局指针：R28

栈指针：R29

帧指针（框架指针）：R30

PC 寄存器: R31

HALT 影响寄存器: R1、R2、R4、R31

不可改写 (值为 0): R0

参数传递寄存器: R4、R5、R6、R7

返回值寄存器: R2、R3

MCR 机器控制寄存器

KBSR 键盘状态寄存器: 提供键盘键入字符的状态信息

KBDR 键盘数据寄存器: 保存由键盘键入的字符的 ASCII 码

DSR 显示器状态寄存器: 提供显示器显示的状态信息

DDR 显示器数据寄存器: 保存将被显示在显示器上的内容的 ASCII 码

PC 程序计数器: 指向下一条指令的地址

IR 指令寄存器: 保存正在处理的指令

MAR 地址寄存器: 用于寻址

MDR 数据寄存器: 用于存取单元内容

CAUSE 原因寄存器: 记录哪些设备发出中断信号 (特权模式允许访问)

SR 状态寄存器: 决定谁能中断处理器 (特权模式允许访问)

EPC 中断时保存 PC

DLX 模拟器

1.配置: dlx 文件夹所在目录配置到环境变量 path 中。

2.汇编: dlxassembler *.dlx 输出*.link

3.链接: dlxlinker *.link *.bin 输出*.bin, 由模拟器运行。